

Re-review: Mqtt_Json_Format_V3.0_Dual_Broker

Date: 02 October 2026 | Repository: REConnect_Firmware_Modules (HEAD 4560abe + uncommitted working-tree changes) | Follows: Code_Review_2026-09-29.pdf

Scope

This review covers the **uncommitted working-tree changes** in the module (11 files, about 4,300 lines added and 2,300 removed, ignoring whitespace): the new dual-broker connection manager in `mqtt_connect.c`, the incoming command queue, transparent mode, date-range FetchDay/GetDay, changes to the DLMS CDF generators (billing, event, LS, midnight), and the new IPsec subnet-mask fields. First, each of the 11 findings from 29 September was re-checked. Then the new code was reviewed for correctness and safety. Nothing was compiled or run; all findings come from reading the source. Line numbers refer to the working tree as of 02 Oct 2026.

Not verified: the DLMS poller that writes the Redis list `mqtt_command_resp` is not in this repository, so the FetchDay response key names could not be cross-checked. The request uses "startdate" while the response handler reads "start_date"; please confirm.

Key takeaways

- The connection-manager rewrite is a clear improvement: sequential connects, clients destroyed only by the worker, stale callbacks ignored, bounded publish waits, and shutdown that joins the worker before destroying clients. Findings 4, 5, 6, 8 and 9 are fixed.
- Findings 1-3 are still open, so MQTT broker set-config and get-config from the server still do not work. These should be fixed first.
- Two new injection issues need attention before release: shell command injection through IPsec fields (N1) and SQL injection through the event category (N2).

Status of the 29 September findings

#	Finding	Status	Notes
1	Set-config looks up Redis field "mqtt1" instead of "primary"	Open	<code>set_cfg_request.c:195</code> still calls <code>rget_int(ctx, tmp, "mqtt1", -1)</code> . <code>main.c:1373</code> (<code>load_mqtt_cfg</code>) also reads field "mqtt1".
2	DATA_TYPE strings do not match the server	Open	<code>set_cfg_request.c:1605/1608</code> still compare against "MQTT_BROKER_mqtt1" / "MQTT_BROKER_mqtt2".
3	Get-config reports wrong field and JSON key	Open	<code>get_cfg_request.c:71</code> still <code>ADD_BOOL("mqtt1", "mqtt1")</code> .
4	Command ACKs published to the wrong topic	Fixed	<code>build_pub_topic()</code> maps <code>CMD_ACK_TOPIC</code> to <code>PUB_ACK_TOPIC</code> .
5	Unbounded wait for publish completion	Fixed	<code>mqtt_publish_blocking()</code> waits at most 10 s (monotonic clock) and returns early if the broker drops.
6	Redis accessed from Paho callback threads	Fixed	Callbacks only set flags; <code>update_mqtt_time()</code> and status writes run on the worker thread.
7	<code>last_message_time</code> updated for both brokers	Open	<code>update_mqtt_time(1)</code> (<code>mqtt_connect.c:4702</code>) still writes <code>last_message_time</code> to both status hashes when both are connected.
8	Cloud LED never turns on	Fixed	Worker loop sets the LED on state change (<code>main.c:994</code>).
9	Commands from two brokers share one buffer slot	Fixed	8-slot mutex-protected queue that records the source broker (<code>mqtt_cmd_enqueue/dequeue</code>).
10	FetchDay responses can go to the wrong broker	Open	<code>Fetchday_cmd_broker</code> is still one global. It is now also set before the DCU serial check (<code>mqtt_connect.c:3886</code>), so a rejected request still redirects pending responses.
11	Modbus JSON device count and export loop disagree	Open	<code>modbus_json_export.c</code> was not changed.

Fixed: 5 Still open: 6 (1, 2, 3, 7, 10, 11). The details and suggested fixes for the open items are unchanged from the 29 September report.

New findings - summary

#	Severity	Location	Issue
N1	High	set_cfg_request.c:351-374; Modem_PPP_IPS EC/ipsec_monitor.sh:340-356	Shell command injection through IPsec fields (incl. new subnet masks)
N2	High	mqtt_connect.c:3647-3680; dlms_cdf_event.c:176/195/202	SQL injection through the event category
N3	High	dlms_cdf_billing.c:901	Multi-month billing compares month but not year
N4	High	dlms_cdf_event.c:328 and :517-527	Event values attributed to the wrong OBIS codes
N5	High	mqtt_connect.c:3510 (parse_getday_cmd)	GetDay load-survey loop is unbounded and sends no heartbeat
N6	Medium	mqtt_connect.c:2628-2860 (read_redis_resp)	FetchDay LS counters are not reset on error paths
N7	Medium	mqtt_connect.c:2917	Uninitialized event_category passed to the event generator
N8	Medium	mqtt_connect.c:2543 vs :2935-2958	Event category 6 returns all events
N9	Medium	mqtt_connect.c:3749 and :2919-2925	FetchDay event dates come from a shared global
N10	Medium	set_cfg_request.c:1578-1583	Transparent mode STOP overwrites met_id
N11	Medium	set_cfg_request.c:1555-1591	Transparent mode START accepted while a session is active
N12	Medium	set_cfg_request.c:1373-1376, :1407	Transparent mode ETHERNET accepts an RS-485 meter
N13	Medium	dlms_cdf_mn.c:331-333, :594/618	Midnight data: empty header and leftover OD table
N14	Low	set_cfg_request.c:126-140 (hset_int_field callers)	Invalid set-config values still report success
N15	Low	set_cfg_request.c:34, :37, :80	Integer conversion without range check
N16	Low	mqtt_connect.c:3844-4063; processServerMsg	Memory leaks and NULL dereference in command handling
N17	Low	file_gen_main.c:2135/2150/2166/2312/2347/2 381	Missing prototypes for changed functions
N18	Low	dlms_cdf_billing.c:996-1000	Billing with no data reported as success
N19	Low	general.h:29; mqtt_connect.c:1720/2635/3737; main.c:1106	128 KB buffers on the stack and chunk size
N20	Low	dlms_cdf_billing.c:1012/1044; dlms_cdf_event.c:695/772	File name copied into 128-byte buffers with strcpy
N21	Low	dlms_cdf_event.c:248	Event query silently capped at 1000 entries
N22	Low	general.h:554 and others	Build hygiene

High: 5 Medium: 8 Low: 9

New findings - details

N1. Shell command injection through IPsec fields (incl. new subnet masks) [High]

Location: set_cfg_request.c:351-374; Modem_PPP_IPSEC/ipsec_monitor.sh:340-356

Problem: The new LEFT_SUBNET_MASK / RIGHT_SUBNET_MASK values (and the existing IPsec strings) are written to Redis without validation. ipsec_monitor.sh loads every field with `eval "cfg_${idx}_x=$(redis_hget ...)"`, so the value is executed by the shell.

Impact: A set-config with "LEFT_SUBNET_MASK": "255.255.255.0; reboot" runs reboot (or any command) as root when ipsec_monitor.sh restarts.

Suggested fix: In C, accept only dotted-quad values for IP/subnet/mask fields and a safe character set for IDs and names; reject the request otherwise. In the script, stop using eval on Redis values (e.g. assign via a temporary variable: `_v=$(redis_hget ...); eval "cfg_${idx}_x=\$_v"`).

Verification: Confirmed

N2. SQL injection through the event category [High]**Location:** mqtt_connect.c:3647-3680; dlms_cdf_event.c:176/195/202**Problem:** GetDay passes the raw server argument cmd.args[4] as event_type, which is inserted into WHERE event_type = '%s' without escaping. Unlike the FetchDay path (mqtt_connect.c:2543), values outside 1-6 are not mapped to "all".**Impact:** A crafted ARG (e.g. 1' OR '1'='1') changes the query on the OD table. Values like 0 or 7 return no rows instead of all events.**Suggested fix:** Accept only 1-6 (else "all") before building the query, and bind the value with sqlite3_bind_text.**Verification:** Confirmed**N3. Multi-month billing compares month but not year [High]****Location:** dlms_cdf_billing.c:901**Problem:** The check if (!is_current_month && month == end_month) multi_month_billing = 0; ignores the year. read_billing_data then drops the OD table (line 196).**Impact:** For a range such as 05_2025..05_2026 the OD table is dropped after the first month; the following months fail silently and the JSON contains only May 2025.**Suggested fix:** Compare year and month, or keep the flag set for the whole loop and drop the OD table once after the loop.**Verification:** Confirmed**N4. Event values attributed to the wrong OBIS codes [High]****Location:** dlms_cdf_event.c:328 and :517-527**Problem:** Each row skips values equal to FFFF/FFFFFFFF, but the PARAMS header is built only from entries[0]. The change widened the skip to FFFF.**Impact:** If row 0 has FFFF in a column and a later row does not, that row has more values than PARAMS and every following value is mapped to the wrong OBIS code (likely with category "all").**Suggested fix:** Do not drop columns per row: emit every column (empty/null for FFFF), or compute one column mask and apply it to the header and all rows.**Verification:** Confirmed**N5. GetDay load-survey loop is unbounded and sends no heartbeat [High]****Location:** mqtt_connect.c:3510 (parse_getday_cmd)**Problem:** The loop generates and publishes one file per day for the whole requested range with no upper limit, and does not call send_hc_msg() or mqtt_conn_manager_poll(). Each publish can wait up to 10 s per chunk.**Impact:** A long date range keeps the worker busy for minutes. dcu_mon_proc restarts re_mqtt_proc after 180 s without heartbeat (MON_HC_TIMEOUT_SEC), killing the request midway; meanwhile no reconnects or cyclic publishing happen.**Suggested fix:** Cap num_days (e.g. 31) and reject larger ranges; call worker_service() after each day.**Verification:** Confirmed**N6. FetchDay LS counters are not reset on error paths [Medium]****Location:** mqtt_connect.c:2628-2860 (read_redis_resp)**Problem:** ls_total_days / ls_completed_days are static. The returns at 2755, 2772 and 2828 leave them (and check_redis_resp) unchanged.**Impact:** The next LS request inherits the old totals: it can finish early, leaving its remaining day responses in mqtt_command_resp to be processed as part of a later request.**Suggested fix:** Reset the counters (and check_redis_resp) on every error path, and key the progress on the request's transaction ID.**Verification:** Confirmed

N7. Uninitialized event_category passed to the event generator [Medium]

Location: mqtt_connect.c:2917

Problem: char event_category[32] is only set when the response contains a string event_type.

Impact: Otherwise stack garbage is passed into the SQL WHERE clause and the output file name.

Suggested fix: Initialise it to "all".

Verification: Confirmed

N8. Event category 6 returns all events [Medium]

Location: mqtt_connect.c:2543 vs :2935-2958

Problem: generate_redis_list accepts category 6 and requests event_data_cat6, but the response mapping only knows cat1-cat5 and falls back to "all".

Impact: A FetchDay for category 6 returns every category.

Suggested fix: Add cat6 to the mapping (or use one shared mapping table for both directions).

Verification: Confirmed

N9. FetchDay event dates come from a shared global [Medium]

Location: mqtt_connect.c:3749 and :2919-2925

Problem: Every processed command does cpy_cmd = cmd. The FetchDay event response handler reads its start/end dates from cpy_cmd.args[2]/[3].

Impact: If any other command (from either broker) arrives between a FetchDay EVENT request and its response, the response is generated with that command's arguments.

Suggested fix: Carry the dates in the Redis request/response JSON (as LS/MN/billing do) instead of the global.

Verification: Confirmed

N10. Transparent mode STOP overwrites met_id [Medium]

Location: set_cfg_request.c:1578-1583

Problem: STOP for ETHERNET writes port, met_id and mode=0. dcu_mon_proc re-reads met_id on exit (dcu_mon_proc.c:2587).

Impact: STOP with a different meter's serial makes dcu_mon restore the wrong ethDaProc; the original one stays stopped until reboot.

Suggested fix: On STOP write only mode=0, or reject STOP if port/met_id do not match the active session.

Verification: Confirmed

N11. Transparent mode START accepted while a session is active [Medium]

Location: set_cfg_request.c:1555-1591

Problem: dcu_mon_proc only starts a session when none is active (dcu_mon_proc.c:2655), but the handler overwrites port, met_id and duration and replies SUCCESS.

Impact: The server is told the second session started although it did not; on exit the wrong process may be restored.

Suggested fix: Read trans_dcu_mode_info mode first and reject START while it is 1 (unless it is the same target).

Verification: Confirmed

N12. Transparent mode ETHERNET accepts an RS-485 meter [Medium]

Location: set_cfg_request.c:1373-1376, :1407

Problem: The met_id lookup by serial ignores the meter's port in the *_details record.

Impact: An RS-485 meter serial with DATA_TYPE=ETHERNET stops an unrelated ethDaProc with the same met_id.

Suggested fix: Also require the meter's port to be the ethernet port index.

Verification: Plausible (lookup confirmed to match on serial only; port layout of *_details not verified)

N13. Midnight data: empty header and leftover OD table [Medium]**Location:** dlms_cdf_mn.c:331-333, :594/:618**Problem:** PARAMS is taken from snapshots[0]; if the first day has no row it is empty while later RECORDS have values. Error returns inside the day loop skip the table drop that only happens on the last day (line 574).**Impact:** Server receives records without parameter names; stale rows from a failed request can leak into the next one.**Suggested fix:** Take the header from the first non-empty snapshot; drop the OD table on error paths too.**Verification:** Confirmed**N14. Invalid set-config values still report success [Low]****Location:** set_cfg_request.c:126-140 (hset_int_field callers)**Problem:** Invalid integers are now skipped (an improvement), but the return value is ignored and set_*_cfg returns 0.**Impact:** The server receives SUCCESS for a value that was not applied.**Suggested fix:** Propagate the -1 and send a failure response; ideally validate all fields before writing any.**Verification:** Confirmed**N15. Integer conversion without range check [Low]****Location:** set_cfg_request.c:34, :37, :80**Problem:** (int) of an out-of-range double is undefined; long to int truncates ("4294967297" becomes 1).**Impact:** Wrong configuration values written to Redis.**Suggested fix:** Check against INT_MIN/INT_MAX and treat as invalid.**Verification:** Confirmed**N16. Memory leaks and NULL dereference in command handling [Low]****Location:** mqtt_connect.c:3844-4063; processServerMsg**Problem:** redis_hget() returns strdup'd memory that is never freed (all branches, including the new transparent-mode one). cmd.root is not freed for GetDay/FetchDay. strcmp(cmd.args[0], dcu_sn) crashes if dcu_info serial_num is missing (all branches except transparent mode).**Impact:** Slow memory growth per command; crash if the serial is missing in Redis.**Suggested fix:** free(dcu_sn) on every path, check it for NULL, and cJSON_Delete(cmd.root) on all returns.**Verification:** Confirmed (pre-existing pattern, extended by new code)**N17. Missing prototypes for changed functions [Low]****Location:** file_gen_main.c:2135/2150/2166/2312/2347/2381**Problem:** generate_billing_json, generate_midnight_json and generate_event_log_json have no prototype in any header and are called through implicit declarations, although their argument lists changed in this diff.**Impact:** A stale caller with the old arguments would compile (only a warning) and corrupt the stack at runtime.**Suggested fix:** Add the prototypes to general.h and build with -Werror=implicit-function-declaration.**Verification:** Confirmed**N18. Billing with no data reported as success [Low]****Location:** dlms_cdf_billing.c:996-1000**Problem:** With no data the code logs an error but still writes a BILLING JSON with empty PARAMS/RECORDS and returns 0 (the return -1 is commented out).**Impact:** Server receives an empty "successful" billing file.**Suggested fix:** Restore return -1 or send an explicit no-data response.**Verification:** Confirmed

N19. 128 KB buffers on the stack and chunk size [Low]

Location: general.h:29; mqtt_connect.c:1720/2635/3737; main.c:1106

Problem: PAYLOAD_BUFFER_SIZE is now 131072 and used for several stack arrays that nest (worker -> processServerMsg -> parse_getday_cmd -> mqtt_send_file): about 256-384 KB per call chain. read_redis_resp zero-fills an unused 128 KB buffer on every call. A 128 KB chunk plus headers is above the 128 KB message limit of some brokers (e.g. AWS IoT Core).

Impact: Fine with the default 8 MB thread stack, but overflows if the stack limit is reduced; publishes may be rejected by brokers with a 128 KB limit.

Suggested fix: Use static or heap buffers, remove the unused buffer in read_redis_resp, and keep chunks below the broker limit.

Verification: Plausible

N20. File name copied into 128-byte buffers with strcpy [Low]

Location: dlms_cdf_billing.c:1012/1044; dlms_cdf_event.c:695/772

Problem: out_path (up to 512 bytes, containing serial, dates and event type from the request/response) is strcpy'd into billing_file_name[128] / event_file_name[128].

Impact: Long inputs overflow the global buffers.

Suggested fix: Pass the destination size and use snprintf.

Verification: Plausible

N21. Event query silently capped at 1000 entries [Low]

Location: dlms_cdf_event.c:248

Problem: The fixed 1000-entry array now also applies to date-range queries.

Impact: Large ranges are cut off without any indication.

Suggested fix: Grow the array dynamically (as billing now does) or log when the cap is reached.

Verification: Confirmed

N22. Build hygiene [Low]

Location: general.h:554 and others

Problem: A static prototype (mqtt_client_error_string) in a shared header causes "declared static but never defined" warnings in every other file. generate_billing_json_single_month and json_write_d6 are now unused. Debug printf output remains in json_write_header ("11111111111") and processServerMsg.

Impact: Warning noise hides real warnings (see N17).

Suggested fix: Move the prototype into mqtt_connect.c, remove dead code and debug prints.

Verification: Confirmed

Recommended order of work

1. Restore the Redis field name "primary" and the server strings MQTT_BROKER_PRIMARY / SECONDARY and key PRIMARY (old findings 1-3), then test set-config and get-config for both brokers against the server.
2. Close the injection paths: validate IPsec fields and remove eval in ipsec_monitor.sh (N1); whitelist/bind the event category (N2).
3. Fix data correctness in the CDF generators: billing year check (N3), event column alignment (N4), midnight header/table cleanup (N13), billing no-data result (N18).
4. Make GetDay/FetchDay robust: cap the date range and keep the heartbeat running (N5), reset LS state on errors (N6), initialise event_category (N7), category 6 mapping (N8), stop using the cpy_cmd global (N9), per-request broker routing (old finding 10).
5. Fix transparent-mode session handling (N10-N12).
6. Remaining items: old findings 7 and 11, then the Low findings (N14-N22).